on

Auth & Inbox Endpoints

Base URL	`https://your-domain.com`

Authentication

All protected routes require a JWT token in the **Authorization** header: `Authorization: Bearer <token>`
Token is returned on login / signup. API-key routes use `?token=` query param instead.

POST `/user/signup`

Register a new user

Parameters

`name`	required	Full name

Response

```
{ "success": true, "msg": "Signup Success" }
```

POST `/user/login`

Login and receive a JWT token

Parameters

`email`	required	Registered email

Response

```
{ "success": true, "token": "eyJ..." }
```

POST `/user/login_with_google`

Login via Google OAuth

Parameters

`token`	required	Google ID token (JWT from Google Sign-In)

Note

Auto-registers user if email is not found. Email and name are decoded from the Google token.

Response

```
{ "success": true, "token": "eyJ..." }
```

POST `/user/login_with_facebook`

Login via Facebook OAuth

Parameters

`token`	required	Facebook access token

Note

Requires FB App ID & Secret configured in admin panel. Token is validated against Facebook's debug endpoint.

Response

```
{ "success": true, "token": "eyJ..." }
```

Inbox — Webhook

GET `/inbox/webhook/:uid` — WhatsApp webhook verification (Meta calls this)

Parameters

`hub.mode` — required — Sent by Meta

Note

:uid = your user UID. Set this URL in Meta App Dashboard under webhook settings.

Response

```
200 — returns hub.challenge (plain text)
```

POST `/inbox/webhook/:uid` — Receive incoming WhatsApp messages & delivery updates

Parameters

`:uid` — required — User UID in path

Note

Always returns HTTP 200 (required by Meta). Internally saves incoming messages, updates delivery status (sent / delivered / read / failed) in beta_api_logs and beta_campaign_logs.

Response

```
HTTP 200 (always)
```

Inbox — Chat Operations

GET `/inbox/get_chats` — List all chats for the authenticated user — auth required

Note

Returns chats merged with contact info. Both tables are joined client-side before returning.

Response

```
{ "success": true, "data": [ ...chats ] }
```

POST `/inbox/get_convo` — Fetch conversation messages for a chat — auth required

Parameters

`chatId` — required — Unique chat ID

Note

Reads last 100 messages from a JSON file on the server at conversations/inbox/{uid}/{chatId}.json

Response

```
{ "success": true, "data": [ ...messages ] }
```

POST `/inbox/send_text` — Send a plain text WhatsApp message — auth required

Parameters

`text` — required — Message content

Response

```
{ "success": true, ... }
```

POST `/inbox/send_image` — Send an image message — auth required

Parameters

`url` — required — Publicly accessible image URL

Response

```
{ "success": true, ... }
```

POST `/inbox/send_audio`

Send an audio message

auth required

Parameters

`url`	required	Publicly accessible audio file URL

Response

```
{ "success": true, ... }
```

POST `/inbox/send_meta_templet`

Send a WhatsApp approved template message

auth required

Parameters

`template`	required	Template object as returned by Meta API

Note

Requires valid Meta API keys saved in profile. Template must be approved by Meta.

Response

```
{ "success": true, "msg": "The templet message was sent" }
```

POST `/inbox/del_chat`

Delete a chat and its conversation history

auth required

Parameters

`chatId`	required	Chat ID to delete

Note

Removes the chat record from DB and deletes the conversation JSON file from disk.

Response

```
{ "success": true, "msg": "Conversation has been deleted" }
```